The Journal of Open Source Software

# Astaroth: A scientific computing framework for accelerating stencil computations.

**Touko Puro** [1¶], **Johannes Pekkilä** [1], **Miikka S. Väisälä** [2], **Oskar Lappi** [3], **Matthias Rheinhardt** [1], **Hsien Shang** [4], **and Maarit Korpi-Lagg** [1]

**1** Aalto University, Finland **2** University of Oulu, Finland **3** University of Helsinki, Finland **4** Academia Sinica Institute of Astronomy and Astrophysics, Taiwan ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:** Patrick Diehl

**Reviewers:**

- @rfj82982
- @streeve

**Submitted:** 20 May 2026
**Published:** unpublished



## Summary

Stencil computations[1] are one of the bedrocks of high-performance scientific simulations, forming the core of many partial differential equation (PDE) and numerical linear algebra solvers. In recent years, GPUs have become the primary compute platform for data-parallel applications in high-performance computing, and it is difficult to run large simulations without them. `Astaroth` is a GPU framework for stencil computations, that has been developed to address this problem of scalable scientific computing. `Astaroth` provides its own domain specific language (DSL), in which researchers can express such computations without having to focus on technical implementation details. It can run efficiently both on CUDA- and HIP-based environments — and even on hardware lacking GPUs, e.g. for testing purposes. While stencils are the core of `Astaroth`, it also accelerates other operations like reductions (e.g. sums), simple ray-tracing, and integrates with libraries performing GPU-accelerated Fourier transforms, all of which are important for simulations on structured grids. `Astaroth` is optimized for multiphysics use cases and has primarily been used for turbulent astrophysical plasma simulations.

## Statement of need

Much of the software used for scientific computing is written for CPUs, and has to be ported to GPUs to run larger problems with decent times-to-solution. `Astaroth` has been developed to solve this problem for the subset of scientific software that relies heavily on stencil computations. `Astaroth`'s domain specific language (DSL) can be used to rewrite existing PDE solvers or to write completely new ones. As an example, `Astaroth` has been used to write a partial differential equation (PDE) solver for astrophysical plasma simulations (Miikka S. Väisälä et al., 2023), which scales to thousands of GPUs with a weak scaling efficiency >90% (Pekkilä, 2026).

Accelerating such simulations was the original reason for the creation of `Astaroth`. A widely used framework for them is the `Pencil Code` (Brandenburg et al., 2020), which is a modular multiphysics PDE and particle dynamics solver. A prototype of `Astaroth` focused on accelerating high-order finite-difference methods for isothermal hydrodynamics, as employed by `Pencil Code`, on GPUs (Pekkilä et al., 2017; M. S. Väisälä, 2017). The `Astaroth` framework for general stencil computations took shape in (Pekkilä, 2019) with the introduction of the DSL, code generator, and multilayer API. With later revisions, `Astaroth` has successfully been used to accelerate `Pencil Code` (Puro, 2023), achieving speedup factors of 20-60 (Pekkilä

[1] Stencil computations are computations on structured grids where a given point is updated using a fixed neighborhood pattern. Examples are convolutions in image processing and convolutional neural networks, and different schemes for spatial derivatives like the finite-difference method.

et al., 2022). Of course, Astaroth's PDE solver is not limited to astrophysics, and neither is Astaroth limited to PDEs. As an example, many image processing techniques, like edge detection and convolutions, are stencil operations.

## State of the field

Methods to achieve performance portability in stencil computations have been widely studied. Domain-specific languages for image processing include Halide (Ragan-Kelley et al., 2013) and Polymage (Mullapudi et al., 2015). Autotuning code-generation frameworks include Patus (Christen et al., 2011) and PARTANS (Lutz et al., 2013).

More generalized software that provides the building blocks for domain-specialized libraries also exist: Delite (Sujeeth et al., 2014) and Lift (Steuwer et al., 2017) provide intermediate languages as targets for domain-specific languages; Kokkos (Trott et al., 2021) and RAJA (Beckingsale et al., 2019) provide abstraction layers for parallel computational patterns but focus on single-node computations; Chapel (Callahan et al., 2004) and Charm++ (Kalé & Krishnan, 1993) provide programming models for parallel and distributed computations; In a more specialized approach, the Cactus framework (Goodale et al., 2003) provides a collection of functionalities shared between computational tasks. We refer the reader to (Pekkilä, 2026) for more details.

Closest to Astaroth is Parthenon (Grete et al., 2023), a distributed framework for adaptive mesh refinement using Kokkos as the backend for intra-node computations. In contrast, Astaroth provides a DSL and an optimizing code generator for implementing the computations akin to Halide, Polymage, and Patus.

A distinctive feature of Astaroth is its specialization for cache-constrained use cases, especially in multiphysics simulations where the values of interdependent fields need to be held in working memory at the same time. Additionally, Astaroth does not only consider stencils in isolation, but also their combinations with other operations inside the same kernel, such as distributed reductions. Astaroth also supports multiple different physics cases.

## Software design

Astaroth consists of three main components: 1) acc, a compiler and runtime system for a domain-specific language (DSL) for stencil computations, 2) an API for executing stencil applications on multi-GPU platforms, and 3) a standalone PDE solver. Below, we present a quick overview of these components. Extensive documentation is available at (The Astaroth developers, 2026).

### acc compiler and runtime system

Astaroth has a DSL for stencil-based computations, designed to be used by domain scientists without having to consider technical implementation details. The main operations, such as stencils, are written in a declarative syntax, and the kernels that use them are written in an imperative syntax.[2] The implementation is left to Astaroth's DSL compiler acc, which applies a number of specialized optimizations. An especially important optimization is the unrolled and reordered computation of all required stencils at the start of the kernels, which enables instruction-level parallelism and efficient usage of caches (Pekkilä, 2026). In addition to stencils, the DSL supports two other operations: 1) multi-GPU reductions – which are commonly needed for stencil-based solvers; and 2) simplified distributed ray-tracing, where rays cannot change directions and are restricted to move through neighbouring grid points – which is necessary for simulations incorporating radiative transfer (Heinemann et al., 2006).

[2] In declarative programming, computations are defined by describing what the results look like; in imperative programming, by describing the steps to perform.

Astaroth's DSL also includes a standard library, providing, inter alia: derivative operators used in PDE solvers, implemented for generally spaced Cartesian, spherical or cylindrical grids; and Poisson solvers, e.g. for self-gravity (Krasnopolsky et al., 2026).

acc transpiles the DSL source into CUDA or HIP source code, which is further compiled into machine code using a native CUDA or HIP compiler. The program thus produced is executed in the acc runtime system, which further optimizes the kernels by autotuning the thread block sizes for kernel execution. acc also supports run-time compilation, because run-time configuration parameters may change the evaluation of conditional statements, thereby changing the branches taken at run-time. With run-time compilation, acc compiles for a given configuration only those parts of the DSL source that will be executed. The information of what code gets executed also allows Astaroth to optimize run-time behaviour more precisely, e.g. memory allocations or communication patterns.

## Multi-GPU runtime system and API

In the DSL, using the ComputeSteps language construct, users can define a list of compute steps specifying a sequence of kernels and boundary conditions. Kernels defined in ComputeSteps may be fused to reduce memory reads. Based on the overall domain decomposition and the stencils' data access patterns, acc infers the dependency relationships between the steps, and constructs a directed acyclic graph (DAG) of dependent tasks. Each step is split into many tasks, one per region: there are 26 smaller regions at each subdomain's boundaries, which are dependent on communicated data from neighbors; and one big region at the core of the subdomain, which is not. Communication tasks are inserted where needed.

Astaroth's task scheduler executes these DAGs, asynchronously launching computation and communication tasks when prerequisite tasks are completed. This improves performance in communication-bound cases, especially for higher process counts (Lappi, 2021). For fast data transfers and to support all possible hardware, both GPU-to-GPU remote direct memory access (RDMA) and CPU-to-CPU communication are supported.

This runtime system can be accessed through Astaroth's runtime API. The API is C-ABI compatible, supporting foreign function interfaces to external applications written in any programming language. The API is organized into two layers: the Device layer and the Grid layer. The Device layer provides access to single-GPU functionality, such as: moving data between CPU and GPU, launching kernels, and loading/storing snapshots from/to disk. The Grid layer provides access to multi-GPU functionality, such as: executing DAGs, distributed initialization, and distributed loading/storing of snapshots. Other special functionality is also provided through the API, such as distributed Fourier transforms.

## Solver

Astaroth also includes a standalone finite-difference PDE solver (Pekkilä et al., 2022), which takes full advantage of the DSL and the multi-GPU API, and can be used to write new simulation models. It also works as a testbed for performance research. This solver uses an astrophysical magnetohydrodynamical setup (`acc-runtime/samples/mhd_modular`) by default, but can be configured to run any DSL code. The samples directory also includes other production-ready setups, e.g. `tfm-mpi` for the test-field method (Pekkilä, 2026).

The solver handles distributed initial conditions, domain decomposition, simulation diagnostics, and logging. It is also designed to react to a number of events, such as NaNs in the simulation data, simulation time limits, and a stop signal given through the file system. The directory `analysis/` contains Python-based data analysis tools, which can be used to process and work with the data produced by the standalone solver.

## Research impact statement

Astaroth has already been used in a number of papers as the core PDE-solver, mainly for astrophysical plasma simulations (Gent et al., 2026; Miikka S. Väisälä et al., 2021, 2023), but also in seismology (Ladino et al., 2025). Additionally, it has been used for research on performance optimization methods(Pekkilä et al., 2017, 2025; Pekkilä, 2026), communication techniques (Lappi, 2021; Pekkilä et al., 2022), compiler techniques (Pekkilä, 2019; Puro, 2023) and other topics (Puro & Pohjonen, 2025; Yokelson et al., 2024). We expect that the recent GPU-acceleration of Pencil Code, which was done by embedding Astaroth's DSL and runtime system into it, will increase the number of Astaroth users. The associated speedup factor of 20-60 will enable more realistic astrophysical simulations in a wide range of use cases from modelling small-scale dynamos (Warnecke et al., 2025) to processes producing primordial gravitational waves and their propagation (Roper Pol et al., 2020).

## Acknowledgements

We acknowledge the contributions of all developers and early users of Astaroth who have been instrumental in its evolution. These include Petr Bém, Jörn Warnecke, Frederick Gent, Ruben Krasnopolsky, Wei-Wen Li, Mordecai Mac Low, Chun-Fan Liu, Man Hei Li, Tzu-Chun Hsu and Indrani Das. We acknowledge the computational resources and services provided by CSC — IT Center for Science, the Aalto Science-IT project, ASIAA High-Performance Computing, and National Center for High-Performance Computing (NCHC), National Applied Research Laboratories (NARLabs) in Taiwan, the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, and resources from LUMI-G through the Euro-HPC joint undertaking. Furthermore, we appreciate the important technical assistance provided by CSC, by people like Fredrik Robertsén and others. The development of Astaroth has received funding from the Academy of Finland, ReSoLVE Centre of Excellence, Grant/Award Number: 307411; The European Research Council, the European Union's Horizon 2020 research and innovation program, project UniSDyn, Grant/Award Number: 818665; KAUTE Foundation, Grant/Award Numbers: 20240173 and 20250154; Research Council of Finland, project MomEnt, Grant/Award Number: 373416. The authors acknowledge support for the CompAS Project from the Institute of Astronomy and Astrophysics, Academia Sinica (ASIAA), the Academia Sinica grant AS-IAIA-114-M01, and the National Science and Technology Council (NSTC) in Taiwan through grants 112-2112-M-001-030, 113-2112-M-001-008, and 114-2112-M-001-001-; the International Collaboration and Cooperation grant for COSMAGG that supports the exchanges between Taiwan and Finland: 113-2927-I-001-513-, 114-2927-I-001-506-, and Research Council of Finland project 359462. MSV thanks the support of Jenny and Antti Wihuri Foundation and Finnish Cultural Foundation in his doctoral thesis work, and therefore the early development in Astaroth prototype.

## AI usage disclosure

AI tools have not been used in any step of software creation, documentation or in the authoring of this paper.

## References

Beckingsale, D. A., Burmark, J., Hornung, R., Jones, H., Killian, W., Kunen, A. J., Pearce, O., Robinson, P., Ryujin, B. S., & Scogland, T. R. (2019). RAJA: Portable performance for large-scale scientific applications. *2019 Ieee/Acm International Workshop on Performance, Portability and Productivity in Hpc (P3hpc)*, 71–81. https://doi.org/10.1109/p3hpc49587.2019.00012

Brandenburg, A., Johansen, A., Bourdin, P. A., Dobler, W., Lyra, W., Rheinhardt, M., Bingert, S., Haugen, N. E. L., Mee, A., Gent, F., & others. (2020). The pencil code, a modular MPI code for partial differential equations and particles: Multipurpose and multiuser-maintained. *arXiv Preprint arXiv:2009.08231*. https://doi.org/10.21105/joss.02807

Callahan, D., Chamberlain, B. L., & Zima, H. P. (2004). The cascade high productivity language. *Proceedings of the Ninth International Workshop on High-Level Parallel Programming Models and Supportive Environments, 2004. Proceedings.*, 52–60. https://doi.org/10.1109/HIPS.2004.1299190

Christen, M., Schenk, O., & Burkhart, H. (2011). PATUS: A code generation and autotuning framework for parallel iterative stencil computations on modern microarchitectures. *Proceedings of the 2011 IEEE International Parallel & Distributed Processing Symposium*, 676–687. https://doi.org/10.1109/IPDPS.2011.70

Gent, F. A., Mac Low, M.-M., Korpi-Lagg, M. J., Puro, T., & Rheinhardt, M. (2026). Asymptotic behaviour of galactic small-scale dynamos at modest magnetic prandtl number. *The Astrophysical Journal Letters*, *1000*(2), L40. https://doi.org/10.3847/2041-8213/ae4c41

Goodale, T., Allen, G., Lanfermann, G., Massó, J., Radke, T., Seidel, E., & Shalf, J. (2003). The cactus framework and toolkit: Design and applications. In G. Goos, J. Hartmanis, J. Van Leeuwen, J. M. L. M. Palma, A. A. Sousa, J. Dongarra, & V. Hernández (Eds.), *High performance computing for computational science — VECPAR 2002* (pp. 197–227). Springer. https://doi.org/10.1007/3-540-36569-9_13

Grete, P., Dolence, J. C., Miller, J. M., Brown, J., Ryan, B., Gaspar, A., Glines, F., Swaminarayan, S., Lippuner, J., Solomon, C. J., Shipman, G., Junghans, C., Holladay, D., Stone, J. M., & Roberts, L. F. (2023). Parthenon—a performance portable block-structured adaptive mesh refinement framework. *The International Journal of High Performance Computing Applications*, *37*(5), 465–486. https://doi.org/10.1177/10943420221143775

Heinemann, T., Dobler, W., Nordlund, Å., & Brandenburg, A. (2006). Radiative transfer in decomposed domains. *Astronomy & Astrophysics*, *448*(2), 731–737. https://doi.org/10.1051/0004-6361:20053120

Kalé, L. V., & Krishnan, S. (1993). CHARM++: A portable concurrent object oriented system based on C++. *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*, 91–108. https://doi.org/10.1145/165854.165874

Krasnopolsky, R., Puro, T., Li, W.-W., Shang, H., Väisälä, M. S., Mac Low, M.-M., Rheinhardt, M., & Korpi-Lagg, M. (2026). Iterative poisson solvers for self-gravity with the GPU code astaroth. *arXiv Preprint arXiv:2605.03563*. https://doi.org/10.48550/arXiv.2605.03563

Ladino, O., Coelho, B. S., Wyzykowski, A. B. V., & Soares, L. (2025). Acoustic wave modeling using the astaroth framework. *Empowering the Energy Shift: The Role of HPC in Sustainable Innovation*, *2025*, 1–3. https://doi.org/10.3997/2214-4609.2025643013

Lappi, O. (2021). A task scheduler for astaroth, the astrophysics simulation framework. *Master's Thesis*. https://urn.fi/URN:NBN:fi-fe2021050428868

Lutz, T., Fensch, C., & Cole, M. (2013). PARTANS: An autotuning framework for stencil computation on multi-GPU systems. *ACM Transactions on Architecture and Code Optimization*, *9*(4), 59:1–59:24. https://doi.org/10.1145/2400682.2400718

Mullapudi, R. T., Vasista, V., & Bondhugula, U. (2015). Polymage: Automatic optimization for image processing pipelines. *ACM SIGARCH Computer Architecture News*, *43*(1), 429–443. https://doi.org/10.1145/2694344.2694364

Pekkilä, J. (2019). Astaroth: A library for stencil computations on graphics processing units.

*Master's Thesis*. https://urn.fi/URN:NBN:fi:aalto-201906233993

Pekkilä, J. (2026). *Graphics processors in high-performance computing: Practice and experience in the simulation of astrophysical magnetohydrodynamics and turbulence* [PhD thesis, Aalto University School of Science]. https://urn.fi/URN:ISBN:978-952-64-3160-4

Pekkilä, J., Lappi, O., Robertsén, F., & Korpi-Lagg, M. J. (2025). Stencil computations on amd and nvidia graphics processors: Performance and tuning strategies. *Concurrency and Computation: Practice and Experience*, *37*(12-14), e70129. https://doi.org/10.1002/cpe.70129

Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Käpylä, P. J., & Anjum, O. (2017). Methods for compressible fluid simulation on GPUs using high-order finite differences. *Computer Physics Communications*, *217*, 11–22. https://doi.org/10.1016/j.cpc.2017.03.011

Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Rheinhardt, M., & Lappi, O. (2022). Scalable communication for high-order stencil computations using CUDA-aware MPI. *Parallel Computing*, *111*, 102904. https://doi.org/10.1016/j.parco.2022.102904

Puro, T. (2023). Programmatic and efficient gpu acceleration of cpu stencil codes. *Master's Thesis*. http://hdl.handle.net/10138/569388

Puro, T., & Pohjonen, A. (2025). GPU acceleration of average gradient method for solving partial differential equations. *Proceedings of the Second SIMS EUROSIM Conference on Modelling and Simulation, SIMS EUROSIM 2024*. https://doi.org/10.3384/ecp212.066

Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., & Amarasinghe, S. (2013). Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, *48*(6), 519–530. https://doi.org/10.1145/2491956.2462176

Roper Pol, A., Mandal, S., Brandenburg, A., Kahniashvili, T., & Kosowsky, A. (2020). Numerical simulations of gravitational waves from early-universe turbulence. *Physical Review D*, *102*(8), 083512. https://doi.org/10.1103/physrevd.102.083512

Steuwer, M., Remmelg, T., & Dubach, C. (2017). LIFT: A functional data-parallel IR for high-performance GPU code generation. *Proceedings of the 2017 IEEE/ACM International Symposium on Code Generation and Optimization*, 74–85. https://doi.org/10.1109/CGO.2017.7863730

Sujeeth, A. K., Brown, K. J., Lee, H., Rompf, T., Chafi, H., Odersky, M., & Olukotun, K. (2014). Delite: A compiler architecture for performance-oriented embedded domain-specific languages. *ACM Transactions on Embedded Computing Systems*, *13*(4s), 1–25. https://doi.org/10.1145/2584665

The Astaroth developers. (2026). *Astaroth documentation*. https://toxpuro.github.io/astaroth/. https://toxpuro.github.io/astaroth

Trott, C. R., Lebrun-Grandié, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R., Harvey, E., Hollman, D. S., Ibanez, D., & others. (2021). Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, *33*(4), 805–817. https://doi.org/10.1109/tpds.2021.3097283

Väisälä, M. S. (2017). *Magnetic phenomena of the interstellar medium in theory and observation* [PhD thesis, University of Helsinki]. https://urn.fi/URN:ISBN:978-951-51-2778-5

Väisälä, Miikka S., Pekkilä, J., Käpylä, M. J., Rheinhardt, M., Shang, H., & Krasnopolsky, R. (2021). Interaction of large-and small-scale dynamos in isotropic turbulent flows from GPU-accelerated simulations. *The Astrophysical Journal*, *907*(2), 83. https://doi.org/10.3847/1538-4357/abceca

Väisälä, Miikka S., Shang, H., Galli, D., Lizano, S., & Krasnopolsky, R. (2023). Exploring the

formation of resistive pseudodisks with the GPU code astaroth. *The Astrophysical Journal*, *959*(1), 32. https://doi.org/10.3847/1538-4357/acfb00

Warnecke, J., Korpi-Lagg, M. J., Rheinhardt, M., Viviani, M., & Prabhu, A. (2025). Small-scale and large-scale dynamos in global convection simulations of solar-like stars. *Astronomy & Astrophysics*, *696*, A93. https://doi.org/10.1051/0004-6361/202451085

Yokelson, D., Lappi, O., Ramesh, S., Väisälä, M. S., Huck, K., Puro, T., Norris, B., Korpi-Lagg, M., Heljanko, K., & Malony, A. D. (2024). SOMA: Observability, monitoring, and in situ analytics for exascale applications. *Concurrency and Computation: Practice and Experience*, *36*(19), e8141. https://doi.org/10.1002/cpe.8141